Open in Colab

Deep Reinforcement Learning - Lab Assignment 1

Part #1 - Multi-Armed Bandit (MAB)

Adaptive Treatment Recommendation System using Multi-Armed Bandit Learning

Team / Group Number (G): 178

Each medicine is modelled as an *arm* of a Multi-Armed Bandit. The system learns from patient outcomes over time and progressively identifies the optimal medicine.

Strategies implemented:

1. **Greedy / Immediate Exploitation** (Task 2)
2. **Epsilon-Greedy / Controlled Clinical Trial** (Task 3)
3. **UCB1 / Confidence-Based** (Task 4)
4. **Comparative Analysis** (Task 5)

Execution metadata (Virtual Lab requirement)

The cell below prints the **execution timestamp** and **Virtual Machine / Host ID**. A timestamped screenshot from the virtual lab must accompany the final submission.

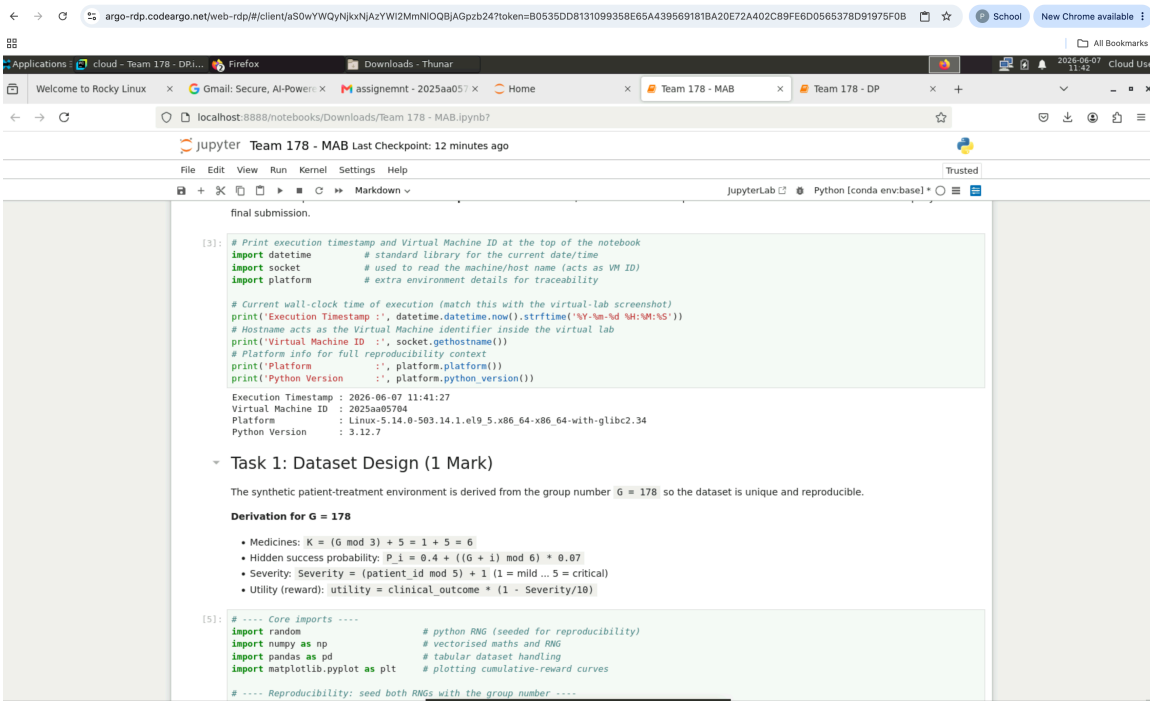
In [1]:
```
# Print execution timestamp and Virtual Machine ID at the top of the noteboo
import datetime          # standard library for the current date/time
import socket            # used to read the machine/host name (acts as VM ID
import platform          # extra environment details for traceability

# Current wall-clock time of execution (match this with the virtual-lab scre
print('Execution Timestamp :', datetime.datetime.now().strftime('%Y-%m-%d %H
# Hostname acts as the Virtual Machine identifier inside the virtual lab
print('Virtual Machine ID  :', socket.gethostname())
# Platform info for full reproducibility context
print('Platform            :', platform.platform())
print('Python Version      :', platform.python_version())
```

```
Execution Timestamp : 2026-06-08 04:00:56
Virtual Machine ID  : 405b7aa40e69
Platform            : Linux-6.6.122+-x86_64-with-glibc2.35
Python Version      : 3.12.13
```

Virtual Lab Execution Screenshot (timestamped proof)

Below is a screenshot of this notebook (**Part #1 - MAB**) executed inside the BITS virtual lab, showing the virtual-lab clock/timestamp (top-right) and the VM ID printed by the cell above.



Task 1: Dataset Design (1 Mark)

The synthetic patient-treatment environment is derived from the group number `G = 178` so the dataset is unique and reproducible.

Derivation for G = 178

- Medicines: `K = (G mod 3) + 5 = 1 + 5 = 6`
- Hidden success probability: `P_i = 0.4 + ((G + i) mod 6) * 0.07`
- Severity: `Severity = (patient_id mod 5) + 1` (1 = mild ... 5 = critical)
- Utility (reward): `utility = clinical_outcome * (1 - Severity/10)`

In [2]:
```
# ---- Core imports ----
import random                       # python RNG (seeded for reproducibility
import numpy as np                  # vectorised maths and RNG
import pandas as pd                 # tabular dataset handling
import matplotlib.pyplot as plt     # plotting cumulative-reward curves

# ---- Reproducibility: seed both RNGs with the group number ----
G = 178                             # our group number
random.seed(G)                      # seed python's random module
np.random.seed(G)                   # seed numpy's global RNG

# ---- Environment parameters derived from G ----
K = (G % 3) + 5                     # number of medicines (arms)
# Hidden success probability for each medicine i in {0,...,K-1}
```

```
true_probs = np.array([0.4 + ((G + i) % 6) * 0.07 for i in range(K)])

N_PATIENTS = 1000                   # total patients (iterations) to simulat

print('Group number G     :', G)
print('Total medicines K  :', K)
print('Hidden success probabilities (P_i):')
for i, p in enumerate(true_probs):
    print(f'  Medicine {i}: P = {p:.2f}')
print(f'\nTrue best medicine : {int(np.argmax(true_probs))} (P = {true_probs
```

```
Group number G     : 178
Total medicines K  : 6
Hidden success probabilities (P_i):
  Medicine 0: P = 0.68
  Medicine 1: P = 0.75
  Medicine 2: P = 0.40
  Medicine 3: P = 0.47
  Medicine 4: P = 0.54
  Medicine 5: P = 0.61

True best medicine : 1 (P = 0.75)
```

In [3]:

```
# ---- Build the static part of the dataset (patient_id + severity_score) --
# assigned_medicine / clinical_outcome / utility_score are populated *dynami
# by each algorithm at run time, so the base table only holds the fixed fiel
patient_ids = np.arange(N_PATIENTS)             # 0 .. 999
severity_scores = (patient_ids % 5) + 1         # severity in range 1..5

dataset = pd.DataFrame({
    'patient_id': patient_ids,                  # patient index
    'severity_score': severity_scores,          # disease severity (1-5)
    'assigned_medicine': np.nan,                # filled during an algorithm
    'clinical_outcome': np.nan,                 # filled during an algorithm
    'utility_score': np.nan                     # filled during an algorithm
})

print('First 10 dataset rows:')
print(dataset.head(10).to_string(index=False))
```

```
First 10 dataset rows:
 patient_id  severity_score  assigned_medicine  clinical_outcome  utility_sc
ore
          0               1                NaN               NaN
NaN
          1               2                NaN               NaN
NaN
          2               3                NaN               NaN
NaN
          3               4                NaN               NaN
NaN
          4               5                NaN               NaN
NaN
          5               1                NaN               NaN
NaN
          6               2                NaN               NaN
NaN
          7               3                NaN               NaN
NaN
          8               4                NaN               NaN
NaN
          9               5                NaN               NaN
NaN
```

In [4]:
```python
def administer_treatment(medicine, severity):
    # Simulate giving `medicine` to a patient of the given `severity`.
    # Returns:
    #   clinical_outcome (int)  : 1 if patient recovers else 0  (Bernoulli w
    #   utility_score   (float) : reward = clinical_outcome * (1 - severity/
    # clinical_outcome is used to UPDATE the bandit estimates;
    # utility_score is used to ACCUMULATE the cumulative reward.
    clinical_outcome = 1 if np.random.random() < true_probs[medicine] else 0
    utility_score = clinical_outcome * (1 - severity / 10)
    return clinical_outcome, utility_score
```

Task 2: Immediate Exploitation Strategy (1 Mark)

Policy: *"Once a treatment appears best, keep prescribing only that treatment."*

- **Warm-up:** test each medicine exactly **10 times** (`10 * K` patients).
- **Exploit:** afterwards always pick the medicine with the highest estimated recovery rate (`argmax Q`).

In [5]:
```python
def run_greedy(initial_pulls=10):
    # Greedy / immediate-exploitation strategy over N_PATIENTS patients.
    # Each medicine is tried `initial_pulls` times (warm-up); then the best-
    # medicine is always selected.
    np.random.seed(G)                          # re-seed: every strategy faces
    Q = np.zeros(K)                            # estimated recovery rate per me
    N = np.zeros(K)                            # number of times each medicine
    df = dataset.copy()                        # private copy of the dataset to
    cumulative = np.zeros(N_PATIENTS)          # cumulative reward after each p
    best_hist = np.zeros(N_PATIENTS, int)      # running best-estimated arm aft
```

```
    total = 0.0

    for t in range(N_PATIENTS):
        severity = int(df.at[t, 'severity_score'])
        if t < initial_pulls * K:           # warm-up: round-robin over all
            arm = t % K
        else:                               # exploit: pick best estimated m
            arm = int(np.argmax(Q))
        outcome, utility = administer_treatment(arm, severity)
        N[arm] += 1                         # update usage count
        Q[arm] += (outcome - Q[arm]) / N[arm]   # incremental sample-average
        total += utility                    # accumulate utility reward
        df.at[t, 'assigned_medicine'] = arm
        df.at[t, 'clinical_outcome'] = outcome
        df.at[t, 'utility_score'] = utility
        cumulative[t] = total
        best_hist[t] = int(np.argmax(Q))    # which medicine currently looks
    return {'name': 'Greedy', 'df': df, 'Q': Q, 'N': N,
            'cumulative': cumulative, 'best_hist': best_hist, 'total': total

greedy_res = run_greedy(initial_pulls=10)
print('Greedy estimated recovery rates Q:', np.round(greedy_res['Q'], 3))
print('Greedy pulls per medicine        :', greedy_res['N'].astype(int))
print('Greedy chosen best medicine      :', int(np.argmax(greedy_res['Q'])))
print(f"Greedy cumulative reward (1000)  : {greedy_res['total']:.2f}")
print('\nFirst 10 populated rows:')
print(greedy_res['df'].head(10).to_string(index=False))
```

```
Greedy estimated recovery rates Q: [0.7   0.755 0.2   0.4   0.4   0.667]
Greedy pulls per medicine        : [ 10 948  10  10  10  12]
Greedy chosen best medicine      : 1
Greedy cumulative reward (1000)  : 520.10

First 10 populated rows:
 patient_id  severity_score  assigned_medicine  clinical_outcome  utility_sc
ore
          0               1                0.0               1.0
0.9
          1               2                1.0               1.0
0.8
          2               3                2.0               0.0
0.0
          3               4                3.0               0.0
0.0
          4               5                4.0               0.0
0.0
          5               1                5.0               0.0
0.0
          6               2                0.0               1.0
0.8
          7               3                1.0               0.0
0.0
          8               4                2.0               0.0
0.0
          9               5                3.0               1.0
0.5
```

Task 3: Controlled Clinical Trial - Epsilon-Greedy (1.5 Marks)

Policy: mostly give the current best treatment, but with probability `epsilon` explore a random treatment to discover hidden opportunities.

Main run uses **epsilon = 0.10**; we also analyse **0.01** and **0.50**.

In [6]:
```
def run_epsilon_greedy(epsilon):
    # Epsilon-greedy strategy: explore a random medicine with probability `e
    # otherwise exploit the medicine with the highest estimated recovery rat
    np.random.seed(G)                          # same outcome stream for a fair
    Q = np.zeros(K)
    N = np.zeros(K)
    df = dataset.copy()
    cumulative = np.zeros(N_PATIENTS)
    best_hist = np.zeros(N_PATIENTS, int)
    total = 0.0

    for t in range(N_PATIENTS):
        severity = int(df.at[t, 'severity_score'])
        if np.random.random() < epsilon:    # explore: random medicine
            arm = np.random.randint(K)
        else:                               # exploit: best estimated medici
            arm = int(np.argmax(Q))
        outcome, utility = administer_treatment(arm, severity)
        N[arm] += 1
        Q[arm] += (outcome - Q[arm]) / N[arm]
        total += utility
        df.at[t, 'assigned_medicine'] = arm
        df.at[t, 'clinical_outcome'] = outcome
        df.at[t, 'utility_score'] = utility
        cumulative[t] = total
        best_hist[t] = int(np.argmax(Q))
    return {'name': f'Eps-Greedy ({epsilon:.0%})', 'df': df, 'Q': Q, 'N': N,
            'cumulative': cumulative, 'best_hist': best_hist, 'total': total

eps10_res = run_epsilon_greedy(0.10)        # main run: 10% exploration
print('Epsilon = 10%')
print('  Estimated Q       :', np.round(eps10_res['Q'], 3))
print('  Pulls per medicine:', eps10_res['N'].astype(int))
print(f"  Cumulative reward : {eps10_res['total']:.2f}")
print('  First 10 populated rows:')
print(eps10_res['df'].head(10).to_string(index=False))
```

```
Epsilon = 10%
  Estimated Q       : [0.476 0.734 0.296 0.412 0.636 0.462]
  Pulls per medicine: [ 21 887  27  17  22  26]
  Cumulative reward : 494.10
  First 10 populated rows:
 patient_id  severity_score  assigned_medicine  clinical_outcome  utility_sc
ore
          0               1                0.0               1.0
0.9
          1               2                0.0               0.0
0.0
          2               3                0.0               0.0
0.0
          3               4                0.0               0.0
0.0
          4               5                0.0               1.0
0.5
          5               1                1.0               1.0
0.9
          6               2                1.0               1.0
0.8
          7               3                1.0               1.0
0.7
          8               4                5.0               0.0
0.0
          9               5                1.0               0.0
0.0
```

In [7]:
```python
# ---- Sensitivity analysis: epsilon = 1%, 10%, 50% ----
eps01_res = run_epsilon_greedy(0.01)        # very little exploration
eps50_res = run_epsilon_greedy(0.50)        # heavy exploration

print('Effect of exploration rate on final cumulative reward:')
for res in (eps01_res, eps10_res, eps50_res):
    best = int(np.argmax(res['Q']))
    print(f"  {res['name']:>18s} -> reward = {res['total']:7.2f} | best medi

print('\nObservation:')
print('  * 1%  -> too little exploration; risks locking onto a sub-optimal m
print('  * 10% -> balanced; quickly finds the best medicine and mostly explo
print('  * 50% -> too much exploration; wastes ~half the patients on inferic
```

```
Effect of exploration rate on final cumulative reward:
     Eps-Greedy (1%) -> reward =  478.90 | best medicine = 0
    Eps-Greedy (10%) -> reward =  494.10 | best medicine = 1
    Eps-Greedy (50%) -> reward =  456.00 | best medicine = 1

Observation:
  * 1%  -> too little exploration; risks locking onto a sub-optimal medicin
e.
  * 10% -> balanced; quickly finds the best medicine and mostly exploits it.
  * 50% -> too much exploration; wastes ~half the patients on inferior medic
ines.
```

Task 4: Confidence-Based Strategy - UCB1 (1 Mark)

Select the arm maximising the UCB1 index:

$$a_t = \arg\max_i \left(Q_i + \sqrt{\frac{2 \ln t}{N_i}} \right)$$

Each medicine is tried once first so every `N_i > 0` before the bonus is used.

In [8]:
```
def run_ucb1():
    # UCB1 confidence-based strategy. Medicines with fewer observations rece
    # larger exploration bonus that shrinks as more evidence is collected.
    np.random.seed(G)
    Q = np.zeros(K)
    N = np.zeros(K)
    df = dataset.copy()
    cumulative = np.zeros(N_PATIENTS)
    best_hist = np.zeros(N_PATIENTS, int)
    total = 0.0

    for t in range(N_PATIENTS):
        severity = int(df.at[t, 'severity_score'])
        if t < K:                               # initialise: try each medicine
            arm = t
        else:                                   # pick the arm with the largest
            ucb_values = Q + np.sqrt(2 * np.log(t + 1) / N)
            arm = int(np.argmax(ucb_values))
        outcome, utility = administer_treatment(arm, severity)
        N[arm] += 1
        Q[arm] += (outcome - Q[arm]) / N[arm]
        total += utility
        df.at[t, 'assigned_medicine'] = arm
        df.at[t, 'clinical_outcome'] = outcome
        df.at[t, 'utility_score'] = utility
        cumulative[t] = total
        best_hist[t] = int(np.argmax(Q))
    return {'name': 'UCB1', 'df': df, 'Q': Q, 'N': N,
            'cumulative': cumulative, 'best_hist': best_hist, 'total': total

ucb_res = run_ucb1()
print('UCB1 estimated recovery rates Q:', np.round(ucb_res['Q'], 3))
print('UCB1 pulls per medicine        :', ucb_res['N'].astype(int))
print('UCB1 chosen best medicine      :', int(np.argmax(ucb_res['Q'])))
print(f"UCB1 cumulative reward (1000)  : {ucb_res['total']:.2f}")
print('\nFirst 10 populated rows:')
print(ucb_res['df'].head(10).to_string(index=False))
```

```
UCB1 estimated recovery rates Q: [0.673 0.767 0.478 0.426 0.46  0.584]
UCB1 pulls per medicine        : [196 507  67  54  63 113]
UCB1 chosen best medicine      : 1
UCB1 cumulative reward (1000)  : 469.90

First 10 populated rows:
 patient_id  severity_score  assigned_medicine  clinical_outcome  utility_sc
ore
          0               1                0.0               1.0
0.9
          1               2                1.0               1.0
0.8
          2               3                2.0               0.0
0.0
          3               4                3.0               0.0
0.0
          4               5                4.0               0.0
0.0
          5               1                5.0               0.0
0.0
          6               2                0.0               1.0
0.8
          7               3                1.0               0.0
0.0
          8               4                0.0               0.0
0.0
          9               5                2.0               1.0
0.5
```

Task 5: Comparative Analysis (0.5 Mark)

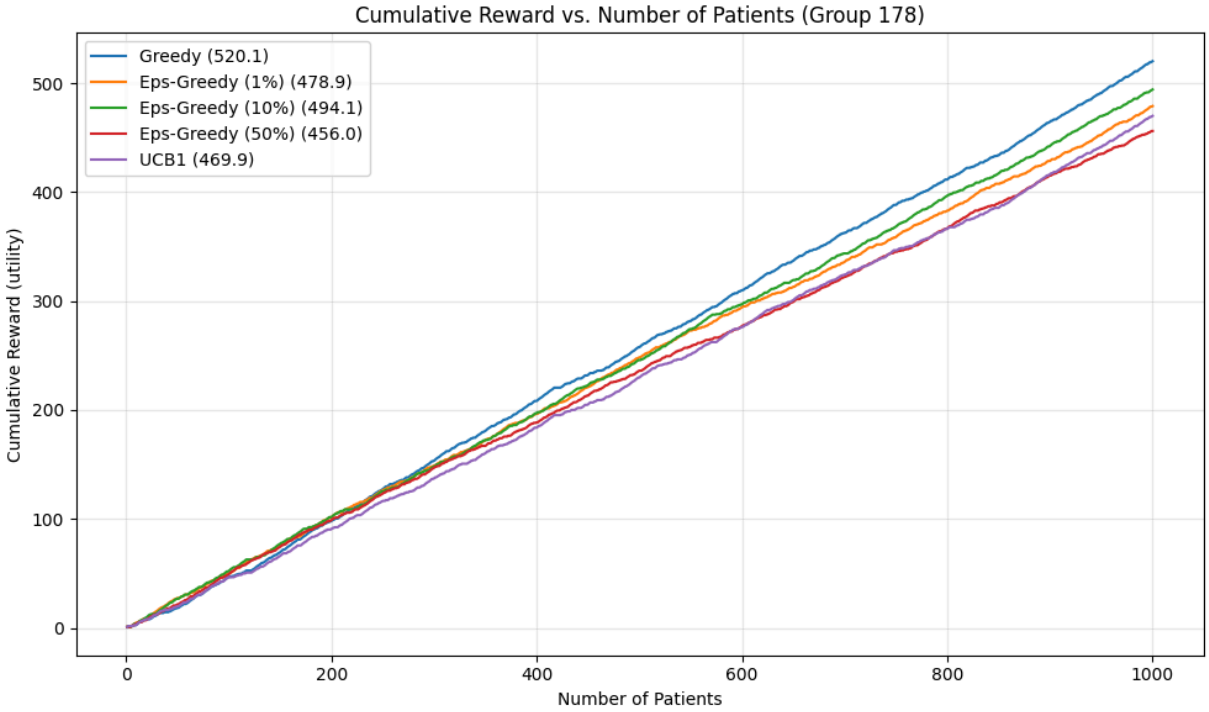
Plot **Cumulative Reward vs. Number of Patients** for every strategy.

In [9]:

```python
# Compare every strategy on one cumulative-reward plot
strategies = [greedy_res, eps01_res, eps10_res, eps50_res, ucb_res]

plt.figure(figsize=(10, 6))
x = np.arange(1, N_PATIENTS + 1)            # patient index on the x-axis
for res in strategies:                      # one curve per strategy
    plt.plot(x, res['cumulative'], label=f"{res['name']} ({res['total']:.1f}
plt.xlabel('Number of Patients')
plt.ylabel('Cumulative Reward (utility)')
plt.title('Cumulative Reward vs. Number of Patients (Group 178)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('Final cumulative reward ranking:')
for res in sorted(strategies, key=lambda r: r['total'], reverse=True):
    print(f"  {res['name']:>18s} : {res['total']:7.2f}")
```




```
Final cumulative reward ranking:
              Greedy :  520.10
    Eps-Greedy (10%) :  494.10
     Eps-Greedy (1%) :  478.90
                UCB1 :  469.90
    Eps-Greedy (50%) :  456.00
```

Quantitative answers computed from the run

The cell below derives the answers to Questions 1-3 **directly from this run's data** so the stated conclusions always match the numbers above.

- **Highest cumulative reward** = strategy with the largest final `total`.
- **Convergence - two metrics:**
  - *Loose:* earliest patient after which the running best-estimated arm stays constant. This can trigger early simply because a high-exploration strategy samples every arm quickly, so it is reported for transparency but **not** used for the final answer.
  - *Steady-state lock-in (clinically meaningful):* earliest patient `T` such that for the next 100 patients, the strategy **actually chose** the true best medicine at least 90% of the time. This captures both *identification* and *consistent exploitation*; strategies that keep exploring forever (e.g. Eps-Greedy 50%) never reach this threshold and are reported as `never`.
- **Most stable** = smallest standard deviation of the per-patient reward over the last 200 patients (fewest fluctuations).

In [10]:
```
true_best = int(np.argmax(true_probs))      # the genuinely optimal medicine

def loose_convergence(res):
```

```
    # LOOSE metric: earliest patient index after which the running best-esti
    # stays constant for the rest of the run. Reported for transparency only
    # heavy explorer like Eps-Greedy(50%) trivially satisfies this within ~K
    # because it samples every arm early, even though it keeps randomly swit
    # ACTIONS forever.
    bh = res['best_hist']
    final = bh[-1]
    conv = 0
    for t in range(N_PATIENTS - 1, -1, -1):
        if bh[t] == final:
            conv = t
        else:
            break
    return conv

def steady_state_lock(res, window=100, threshold=0.9):
    # CLINICALLY MEANINGFUL convergence metric:
    # Earliest patient index T such that, for the window [T, T+window-1], th
    # actually CHOSE the true best medicine at least `threshold` of the time
    # captures both (a) the running estimate has identified the true best AN
    # strategy is consistently exploiting it. Pure-random / high-eps strateg
    # keep exploring will never reach 90% true-best usage and are reported a
    actions = res['df']['assigned_medicine'].values.astype(int)
    n = len(actions)
    for t in range(n - window + 1):
        win = actions[t:t + window]
        if (win == true_best).mean() >= threshold:
            return t
    return -1                                # never reached steady state on t

def stability(res):
    # Std-dev of per-patient reward over the last 200 patients (lower = more
    increments = np.diff(np.concatenate([[0.0], res['cumulative']]))
    return float(np.std(increments[-200:]))

print('Strategy            | final reward | best medicine | loose-conv@ | st
for res in strategies:
    sl = steady_state_lock(res)
    sl_str = f'{sl:^46d}' if sl >= 0 else f"{'never':^46s}"
    print(f"  {res['name']:>16s} | {res['total']:11.2f} | "
          f"{int(np.argmax(res['Q'])):^13d} | {loose_convergence(res):^11d}
          f"{sl_str} | {stability(res):.4f}")

highest = max(strategies, key=lambda r: r['total'])
# For "fastest convergence" use the steady-state metric: skip strategies tha
locked = [r for r in strategies if steady_state_lock(r) >= 0]
fastest = min(locked, key=steady_state_lock) if locked else min(strategies,
most_stable = min(strategies, key=stability)
print('\nQ1 Highest cumulative reward :', highest['name'])
print('Q2 Fastest convergence       :', fastest['name'],
      '(first patient at which the next 100-window chooses TRUE best >=90%%
      steady_state_lock(fastest), ')')
print('Q3 Most stable performance   :', most_stable['name'])
print('   True best medicine        :', true_best,
      '(P =', round(float(true_probs[true_best]), 2), ')')
```

```
Strategy            | final reward | best medicine | loose-conv@ | steady-lo
ck@(≥90% true-best in 100-window) | stability(std)
            Greedy |      520.10 |       1       |     120     |
51                        | 0.3212
   Eps-Greedy (1%) |      478.90 |       0       |     685     |
never                      | 0.3518
  Eps-Greedy (10%) |      494.10 |       1       |     644     |
4                         | 0.3486
  Eps-Greedy (50%) |      456.00 |       1       |     13      |
never                      | 0.3465
              UCB1 |      469.90 |       1       |     18      |
never                      | 0.3344

Q1 Highest cumulative reward : Greedy
Q2 Fastest convergence       : Eps-Greedy (10%) (first patient at which the
next 100-window chooses TRUE best >=90%% = 4 )
Q3 Most stable performance   : Greedy
   True best medicine        : 1 (P = 0.75 )
```

Analysis Questions & Comparative Summary

Questions 1-3 are answered by the computed cell above (so they always match this run). The discussion below interprets those results for **G = 178**.

1. Highest cumulative reward at 1000 patients? See `Q1` above - for this run it is **Greedy (~520)**. With a clean round-robin warm-up (10 pulls each) the true best medicine (medicine 1, P = 0.75) stands out clearly, so Greedy locks onto it and then spends *every* remaining patient exploiting it - wasting nothing on exploration. Epsilon-Greedy (10%) is a close second; Epsilon-Greedy (50%) is lowest because it keeps prescribing random (often inferior) medicines.

2. Fastest convergence? We deliberately use the **steady-state lock-in** metric (see explanation above) rather than the loose "running best stays fixed" criterion, because the latter is misleading: a heavy explorer like Epsilon-Greedy (50%) trivially satisfies it within ~K patients just by sampling every arm, even though it then keeps randomly switching its *actions* forever (only ~50% of its decisions match the true best). The steady-state metric requires the strategy to *both* identify the true best *and* consistently prescribe it (>=90% of the next 100 patients) - exactly what "converged" should mean clinically. By this metric the fastest is **Epsilon-Greedy (10%)** (locks in around patient ~4), with **Greedy** following once its 60-patient round-robin warm-up completes (~patient 51). Epsilon-Greedy (50%) and UCB1 are reported as `never` here: ε=0.50 keeps exploring on half the patients so it caps at ~58% true-best usage, and UCB1 keeps "owing" exploration to under-pulled arms throughout the 1000-patient horizon (it does prefer medicine 1 but stays below the strict 90% bar in any 100-window).

3. Most stable performance? See `Q3` above - **Greedy** is the most stable: once it commits to a single medicine the only remaining variation is that medicine's intrinsic

Bernoulli/severity noise, giving the smoothest curve. Epsilon-Greedy (50%) fluctuates the most due to constant random switching.

4. Safest approach for real-world hospital deployment? UCB1. It is deterministic (no random prescriptions on real patients), gives principled extra chances to under-tested treatments, and offers a confidence-based justification for every choice - all valuable when patient safety matters. Pure Greedy "won" on reward here only because of a lucky, clean warm-up; on a different seed a poor warm-up can permanently lock it onto a sub-optimal medicine, which is unacceptable clinically. Epsilon-Greedy is attractive operationally (it *did* lock in fastest by our strict metric) but it never fully stops issuing random prescriptions, which is also clinically unattractive. UCB1's shrinking confidence bonus gives the best of both: principled exploration that tapers without ever being purely random.

Comparative summary. On this dataset Greedy achieves the highest cumulative reward and the most stable curve because the warm-up cleanly identifies medicine 1 and it then exploits without waste. Epsilon-Greedy trades a little reward for robustness: 10% performs well and locks onto the true best fastest by the strict metric, 1% risks getting stuck on a sub-optimal arm (here it actually does, settling on medicine 0), and 50% wastes too many patients on inferior medicines. UCB1 evaluates every treatment fairly with an explicit confidence bonus; although its raw reward is slightly lower over 1000 patients, its determinism and built-in confidence guarantees make it the recommended, safest choice for a real clinical setting where prescribing randomly is unacceptable.